

Research Computing: Coursework Report

Building and Distributing Dual Autodifferentiation Package

Author: Ilya Kogan
Data Intensive Science
Department of Physics, University of Cambridge



December 18, 2024

Word Count: 2801

1 Introduction

This report discusses the creation of a Python package that implements forward-mode automatic differentiation using dual numbers.

A dual number is an expression of the form $a + b\epsilon$ where $a, b \in \mathbb{R}$ and $\epsilon^2 = 0$. One can note that for a dual number $x = a + b\epsilon$, $x^2 = (a + b\epsilon)^2 = a^2 + 2ab\epsilon + b^2\epsilon^2 = a^2 + 2ab\epsilon$, using the definition of the dual number, namely that $\epsilon^2 = 0$. Also, the derivative of x^2 is $2x = 2a + 2b\epsilon$. Note that with $b = 1$ we get $x^2 = a^2 + 2a\epsilon$ where the dual part is the derivative of the real part. More generally, any function $f(x)$ can be extended to dual numbers space $f(a + b\epsilon) = f(a) + f'(a)b\epsilon$ with the dual part representing the derivative of the function.^[1] This is the property that allows for automatic differentiation which is heavily implemented in the Neural Networks as it enables to compute the derivatives, including high-order derivatives, with respect to many inputs at working precision in a timely manner.^[2]

2 Creating project repository structure

To start the creation of the Python package, we create a new folder, our project repository, and name it `dual_autodiff_pkg`. An appropriate structure of the repository is crucial for the computer to recognize it as a package. For instance, it must include `pyproject.toml` file which contains project configuration, `README.md` markdown file which contains instructions on how to install and use the package, and sub-folders named `dist`, `docs`, `tests` which will contain distribution, documentation and test files, respectively. Most importantly, however, it must include a sub-folder named exactly like we want to name our package, `dual_autodiff`, with Python files: it must contain a `__init__.py` file which signals to Python that it is dealing with a package ^[3], a code file which we name `dual.py` and a `version.py` file which handles the dynamic versioning of our package with Git.

We also include a couple of hidden files: `.gitignore` where we outline which files to exclude from pushing to the git repository (e.g. GitLab) and `.readthedocs.yaml` which contains Read the Docs configuration for our documentation. Overall, we can visualize the core structure of our package as demonstrated in Figure 1.

```

.
├── .gitignore
├── .readthedocs.yaml
├── README.md
├── dist
├── docs
├── dual_autodiff
│   ├── __init__.py
│   ├── dual.py
│   └── version.py
├── pyproject.toml
└── tests

```

Figure 1: This was obtained by running the `tree -a -L 3` command in Terminal from within the project repository (`dual_autodiff_pkg`). It demonstrates the structure of our package which includes files and directories as described above.

3 Project configuration

We proceed with writing the project configuration in the `pyproject.toml` file. It specifies the build-system and project metadata: dependencies, requirements and other information for PyPi.

The build-system table contains a "build-backend" key, which specifies the build backend to be used. [4] It also contains a "requires" key, which lists dependencies needed to build the project. [4] This ensures that all dependencies are either already available or are installed during package installation in the target environment.

The rest of the file describes the project metadata, also in form of a table, where most values are hard coded, but some are passed to the build backend to take care of, like we do with the version control of our package by using `dynamic = ["version"]` which fills the version from `git tag`. [4]

The dependencies are other packages that we import at the beginning of our code file and are required for our package to function. For this package, the only dependency is the `numpy` package. We, also, set a Python version requirement for our file, `requires-python = ">=3.9"`.

Other information in `pyproject.toml` includes the package name (which would appear on PyPi if uploaded), one-line description of the package purpose (portrayed as the headline on PyPi), keywords to aid PyPi's search to suggest our package when these keywords are queried, the author's name, and a link to `README.md` file for the builder to extract information to the PyPi page. [4] Moreover, classifiers contain additional information for package's visibility on PyPi: Development Status corresponds to the maturity of the project, Intended Audience can be specified, as well as the license can be specified here if it is standard and well-known, like MIT license. [4]

Additionally, `project.urls` specifies the URLs associated with the project, `tool.setuptools_scm` section configures the `setuptools_scm` for versioning, and `where = ["."]` specifies the directory to search for the package, which in our case (.) means the current directory. [4]

4 Dual numbers implementation

The dual numbers are implemented in the `dual.py` file in the `Dual` class where mathematical operations using dual numbers are also defined. The file begins with the import of the necessary dependencies (in this example, it is `numpy`), which must match those listed in `pyproject.toml`. However, if the dependency is a part of Python's standard library, like `math`, it should not be included in the `pyproject.toml` because it is available by default in Python installations. Nevertheless, it still needs to be imported in `dual.py` to be used (so we add `import math`).

We define our `Dual` class starting with the initialization method, which is required to create an instance of a class: `__init__(self, real, dual)`. The `self` parameter is a reference to the instance of the class and must be there; the `real` and `dual` parameters are used to set the attributes of the instance. [3] This method takes 2 numbers (integers or floats) as inputs for real and dual parts and outputs a dual number which is an instance of `Dual` class, for example:

```
>>> Dual(2, 1)
Dual(2, 1)
```

We also define `__repr__(self)` method which defines the representation of our class when printed in Python. This is the reason why the output of the above command is `Dual(2, 1)` and not `dual_autodiff.dual.Dual object at 0x1050d3c50` which would be a default object representation in Python that prints object's Class name and memory location.

Next we define `__add__(self, other)`, `__sub__(self, other)`, `__mul__(self, other)`, `__truediv__(self, other)` methods that override the default implementations of `+`, `-`, `*`, and `/` so that when used with `Dual` instances the operations are performed correctly. While addition and subtraction are straight-forward and are done component-wise, i.e. $(a + b\epsilon) \pm (c + d\epsilon) = (a \pm c) + (b \pm d)\epsilon$, multiplication is implemented using the derived formula: $(a + b\epsilon) * (c + d\epsilon) = ac + ad\epsilon + bc\epsilon + bd\epsilon^2 = ac + (ad + bc)\epsilon$. The division formula is derived by using a dual conjugate, defined analogously to complex conjugate: for a dual number $(a + b\epsilon)$, its conjugate is defined as $(a - b\epsilon)$. Using this, we derive the division formula for dual numbers: $(a + b\epsilon) \div (c + d\epsilon) = \frac{a + b\epsilon}{c + d\epsilon} * \frac{c - d\epsilon}{c - d\epsilon} = \frac{ac - ad\epsilon + bc\epsilon}{c^2 - d^2\epsilon^2} = \frac{a}{c} + \frac{bc - ad}{c^2}\epsilon$.

An important note is that each of the methods described checks whether the `other` input is also a `Dual` instance to prevent non-sense operations. Moreover, the initialization method verifies that the parameters for the real and dual parts are numbers. If the checks fail, user encounters a `TypeError`, which signals of an inappropriate type usage and provides specific details of an error. We have also decided to add a small enhancement that allows for commands, like `Dual('1', 1)`, to be interpreted as `Dual(1, 1)` while issuing a warning statement to the user.

Other implemented functions include sine, cosine, tangent, logarithm, exponentiation, and power. The formula for each was derived from an earlier mentioned fact that $f(a + b\epsilon) = f(a) + f'(a)b\epsilon$. [1] For example, the natural logarithm of the dual number is $\log(a + b\epsilon) = \log(a) + \frac{d}{da}\log(a)b\epsilon = \log(a) + \frac{b\epsilon}{a}$, the

cosine of a dual number is $\cos(a + b\epsilon) = \cos(a) - \sin(a)b\epsilon$ as $\frac{d}{dx}\cos x = -\sin x$.

Furthermore, we define a `__pow__(self, other)` method which overrides the power operator, `**`. The formula for raising dual numbers to real powers was derived in the same way: $(a + b\epsilon)^n = a^n + na^{n-1}b\epsilon$ since $\frac{d}{dx}x^n = nx^{n-1}$. However, to derive the formula for raising dual numbers to dual powers, $(a + b\epsilon)^{(c+d\epsilon)}$, we used a logarithm and an exponential: $(a + b\epsilon)^{(c+d\epsilon)} = \exp[\log((a + b\epsilon)^{(c+d\epsilon)})] = \exp[(c+d\epsilon)\log(a + b\epsilon)] = \exp[(c+d\epsilon)(\log(a) + \frac{b}{a}\epsilon)] = \exp[c\log(a) + d\log(a)\epsilon + \frac{cb}{a}\epsilon + \frac{db}{a}\epsilon^2] = \exp[c\log(a) + (d\log(a) + \frac{cb}{a})\epsilon] = a^c + a^c(d\log(a) + \frac{cb}{a})\epsilon$. Hence, our power method first determines whether a dual number is being raised to dual or real power and then performs the operation accordingly.

5 Package initialization and installation

The basis of our package is prepared, we proceed to initialize and install it to test and perform a differentiation example. To setup the dynamic versioning and development workflow, we begin by running `git init` in terminal from inside the package folder, which creates `.git` repository that contains necessary files for version control; then using `git add .` we stage all the files in the folder for Git; then commit changes with the message "Initial commit" using `git commit -m "Initial commit"` and label it with `git tag 0.0.0beta0`. [3]

To install the package in editable mode, we run `pip install -e .` from the command line inside the package folder (`dual_autodiff_pkg`). We are now able to import the package into a Python session and start using it! To do so, begin a Python session using `python` and run `import dual_autodiff as df`, then we will be to use the package as follows: `x = df.Dual(3,4)`. Alternatively, run `from dual_autodiff import Dual`, then the package can be used like this: `x = Dual(3,4)`.

6 Function differentiation example

We now use our package to perform an example differentiation of the function $f(x) = \log(\sin(x)) + x^2\cos(x)$. We first compute the derivative analytically using chain and product rules: $\frac{d}{dx}f(x) = \frac{(\sin x)'}{\sin x} + (x^2)' \cos x + x^2(\cos x)' = \frac{\cos x}{\sin x} + 2x\cos x - x^2\sin x$. Substituting $x = 1.5$ into the derivative formula, we find that $(f'(x)|x = 1.5) = -1.9612372705533612$.

We, then, compute the derivative using our new `dual_autodiff` package. To do so, all we do is calculate $f(x) = \log(\sin(x)) + x^2\cos(x)$ at $x = 1.5 + 1\epsilon$ because the dual part of the $f(1.5 + 1\epsilon)$ would be exactly $f'(1.5)$ as seen before (here:1). We get $f(1.5 + 1\epsilon) = 0.15665054756073515 - 1.9612372705533612\epsilon$ where the dual part is exactly equal to the analytic solution at machine precision.

Additionally, we compute this derivative numerically with an increasingly smaller step h using the definition of the derivative: $\frac{f(x_0+h)-f(x_0)}{h}$ at $x_0 = 1.5$. As we can see in Figure 2, the error of the numerical derivative gets smaller as the step size (h) decreases, which is the expected behavior. The bump at step

size of 10^{-10} can be explained by the overflow (floating-point precision error) that seems to occur at that point. So while smaller step sizes provide more accurate results, if a step size becomes too small, it leads to numerical errors.

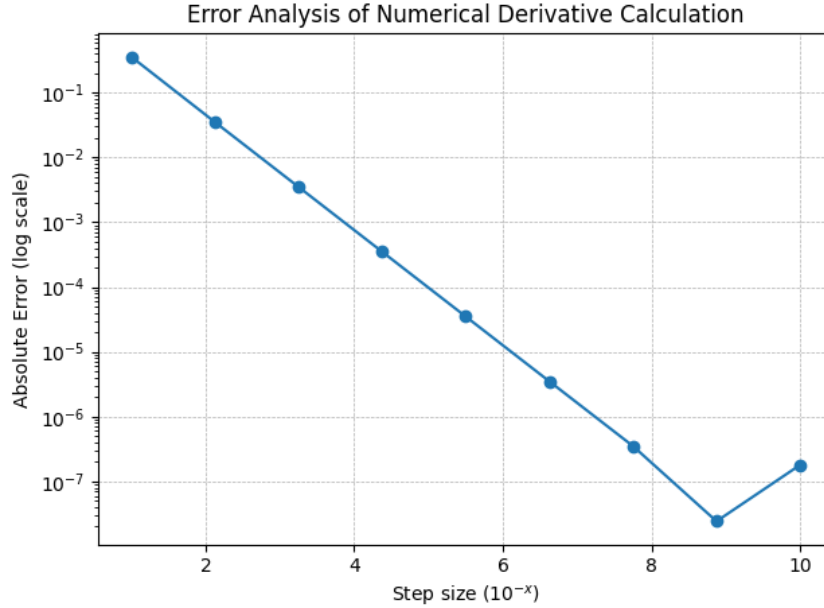


Figure 2: Demonstration of the absolute error of numerical derivative compared to the dual or analytical derivative at $x=1.5$ for increasingly smaller step sizes. While smaller step sizes provide more accurate results, a step size too small leads to numerical errors.

7 Test Suite (pytest)

We now implement a comprehensive test suite for the Dual class in the tests folder. Test suites are very useful when it comes to package development as it makes easy to ensure proper functionality when adding new methods. They are also important when it comes to package distribution as they can verify that the package works correctly upon installation.

Since we have only one code file, `dual.py`, our test suite will consist only of `test_dual.py` where we use `assert` statements to verify that desired behavior is occurring when running different commands. We check that operations are performed according to derived formulas, e.g. `Dual(2,1)*Dual(3,2) = Dual(6,7)` and `Dual(2, 1).log() = Dual(log(2),0.5)` where `log(2)` is a natural logarithm of 2. Furthermore, we verify that exceptions are raised as expected, for instance, that `Dual(0.5π, 1).tan()` raises `ZeroDivisionError` as the dual part of this is equal to the derivative of tangent evaluated at 0.5π and $\frac{d}{dx} \tan x = \frac{1}{\cos^2 x}$ while $\cos(0.5\pi) = 0$.

To run the test suite, we install the pytest module using `pip install pytest`.

Then, we run `pytest -s tests/*` from within our project folder and expect to see an output as demonstrated in Figure 3.

```
configfile: pyproject.toml
collected 26 items

tests/test_dual.py .WARNING: You should use an int or float as input for Dual nu
mber
.WARNING: You should use an int or float as input for Dual number
.....

===== 26 passed in 0.06s =====
```

Figure 3: Output after successful completion of all 26 tests from `test_dual.py` (including 2 when converting '1' into numbers resulting in custom-made Warning statements).

8 Project Documentation (Sphinx)

To build the documentation for our package, we use Sphinx: an automatic documentation generation tool. We write detailed docstrings in our code file `dual.py` for every method. These docstrings describe what each method achieves, its parameters, and return values. Sphinx then generates the documentation from these docstrings. Not only this automates the documentation process, but it also enhances code readability by providing in-line documentation.

We `pip install Sphinx` and use `sphinx-quickstart` from within our `docs` folder to build initial files necessary for Sphinx to create the documentation. We adapt them based on the example provided in `company_package` [5]. Automatically created files are: `conf.py` which contains the configuration of the documentation and some metadata about the project, `index.rst` which is the main page of the documentation, written in reStructuredText, and the `Makefile` which contains the commands to build the documentation. [3]

We also add `tutorial_notebook.rst` file, which includes the Jupyter notebook page that provides some examples of using the package, and `comparison.rst` file which makes a page with the comparison of numeric and dual differentiation approaches, discussed in Section 6.

We needed to install a few Sphinx extensions as `make html` command that we use to build html documentation resulted in errors, like: Extension error: Could not import extension nbsphinx (exception: No module named 'nbsphinx'), which was easily fixed by running `pip install nbsphinx`. Upon successful execution of `make html` command from within the `docs` folder, a new directory named `_build` appears where we can find an `html` folder with generated web pages in html format which contain our documentation. The main page is in the `index.html` file.

9 Cython

We have now created a Python package with documentation and test suite. While it is possible to distribute it already as a pure Python package, we can op-

timize its performance using Cython. In general, Cython speeds up the Python code thanks to more efficient handling of variables and execution: for instance, Cython arrays are static, meaning that the type is fixed (unlike in Python), which eliminates the need for type checks and, hence, speeds up the code. [3]

To Cythonize our package, we begin by installing Cython, using `pip install Cython`. Then we create a new repository `dual_autodiff_x`, to which we copy our core files, `__init__.py` and `dual.py`, into a new sub-folder `src/dual_autodiff_x`. While it is possible to directly convert the Python package into a Cython package, for this project, we preserve the original files by making copies of them in a different directory. We then rename the copied files to `__init__.pyx` and `dual.pyx`, respectively. We copy `pyproject.toml` file to `dual_autodiff_x` and modify it by changing the name of the package to `dual_autodiff_x`, updating the build system to

```
[build-system]
requires = ["setuptools", "Cython", "wheel"]
build-backend = "setuptools.build_meta"}
```

as well as adding relative path to our files

```
[tool.setuptools]
package-dir = {"" = "src"}
```

[3]. We, also, create a `setup.py` file next to the `pyproject.toml` with additional build instructions for Cython extensions. This file is adapted for our package `dual_autodiff_x` based on the `pygbm_x` example.[6] Overall, we should have the structure as demonstrated in Figure 4.

```

├── pyproject.toml
├── setup.py
└── src
    └── dual_autodiff_x
        ├── __init__.pyx
        └── dual.pyx

```

3 directories, 4 files

Figure 4: Output after running `tree -L 3` command from our new directory `dual_autodiff_x` where we have prepared files as described above to use Cython

To build the package we run `python setup.py build_ext --inplace` which creates `dual.C` and `.so` files. Now we can use `python setup.py bdist_wheel` which creates a wheel inside a `dist` folder.[3] This wheel can then be installed using `pip install` command with the location and name of the wheel, in our case: `pip install dist/dual_autodiff_x-0.1.0rc0-cp313-cp313-macosx_14_0_arm64.whl` [3].

Note that this will install cythonized package, but not in editable mode as we simply copied pre-compiled files contained in the wheel into our environment. However, we can also install this package in editable mode by running `pip install -e .` as before.

10 Python vs Cython performance

We now have both pure Python and Cython packages installed. We compare their performances in `dual_autodiff.ipynb` notebook. As illustrated in Figure 5, Cython code is usually executed faster. Additionally, we notice that some operations, like tangent and power, take more time to perform compared to other operations. It is expected, however, as both functions have more mathematical operations in their implementations than others.

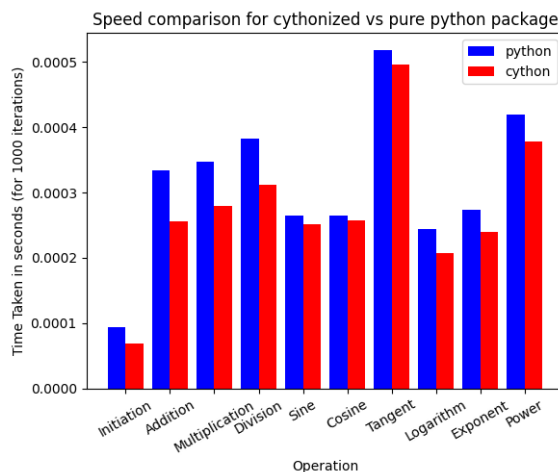


Figure 5: Time taken in seconds to perform each of the operations in Cython and Python 1000 times on a specific local machine.

While absolute values might vary for different machines, respective ratios will be very similar. Hence, we examine the time ratios and find that in Cython it takes, on average, 85% of the time to perform the same operation compared to Python as illustrated in Figure 6. Obviously, as number of operations accumulates, the advantage of Cython's performance over Python's becomes more evident.

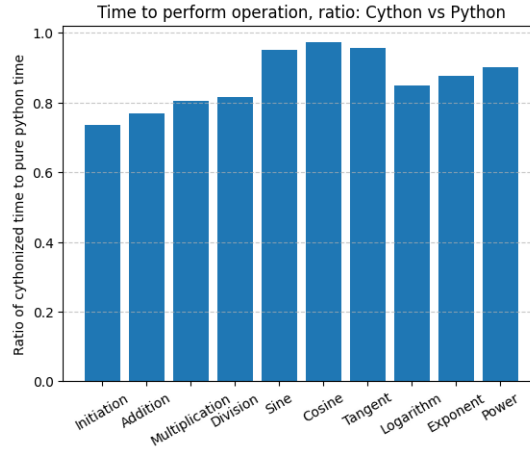


Figure 6: Ratio of execution times for Cython operations relative to Python operations, showing performance improvements.

11 Package distribution

For package distribution, we create wheels which are a binary distribution of the package.[3] One of the benefits is that, "installing from the wheel can be much faster than from the source". [3] Moreover, wheels will already contain pre-compiled files for packages with compiled extensions, further reducing installation time and complexity. [3]

To create specific wheels for `dual_autodiff_x` on Linux we use `cibuildwheel` which we install with `pip install cibuildwheel`. We then build the wheel with

```
CIBW_BUILD="cp311-manylinux_x86_64" CIBW_ARCHS="x86_64" cibuildwheel --platform linux
```

from the root of the package while keeping open the Docker Desktop application (as Mac is being used). We expect output like shown in Figure 7, meaning that we successfully built the specified wheel in `wheelhouse` sub-directory.

```
1 wheels produced in 0 minutes:
  dual_autodiff_x-0.1.0rc0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_6
4.whl 352 KB
```

Figure 7: Last lines of Terminal output, demonstrating successful built of the wheel.

After that we run

```
unzip wheelhouse/<name of wheel>.whl -d wheel_contents
```

with `<name of the wheel>` being the name of the produced wheel, to verify that our wheel does not contain source code files (`.pyx`). We repeat this procedure to build the `cp310` wheel.

To distribute our package, we can simply upload these wheels to PyPi along with our documentation and installation instructions allowing users to easily download and install our package. In such scenario, we would also upload wheels for other operating systems to cover as many platforms as possible.

12 GitLab upload

For the purpose of this coursework, however, we upload our package and wheels to GitLab repository, following these steps:

1. We clone the dedicated GitLab project repository to our local machine, using `git clone <link of repository>`.
2. We add our `dual.autodiff_pkg` and `dual.autodiff_x` folders containing our 2 packages, their documentation, Jupyter notebooks, and wheels to this repository. We move `.gitignore` and `README.md` files to the outmost folder.
3. It seemed like we only needed to add, commit, and push these changes to GitLab, but we faced an issue of git tracking since our folders were initialized as git repositories themselves. We tried following Terminal's suggestions by creating submodules, but then these folders weren't visible on GitLab
4. To resolve the issue of git tracking, we removed `.git` folder from `dual.autodiff_pkg` and `dual.autodiff_x` (making them not tracked by git anymore)
5. This resulted in dynamic version handling issues, so we removed initialization of dynamic versioning from `pyproject.toml` files and hard-coded version numbers, which resolved the issues. Now `pip install -e .` functions properly for our packages once again.
6. We then were able to stage all files with `git add .`
7. Commit changes with a message `git commit -m "package ready"`
8. And push everything to the remote server `git push`

13 Summary

This report describes the process of creation of a Python package that implements automatic differentiation using Dual numbers in great detail. It covers the implementation of Dual Class which handles dual numbers appropriately and the use of Cython and its ability to improve package performance. Additionally, it describes the process of package distribution to different platforms using `cibuildwheel`. This report follows standard good practices of software development throughout the process.

References

- [1] Wikipedia contributors. *Dual number*. Accessed: 2024-12-12. 2024. URL: https://en.wikipedia.org/wiki/Dual_number.
- [2] Wikipedia contributors. *Automatic differentiation*. Accessed: 2024-12-12. 2024. URL: https://en.wikipedia.org/wiki/Automatic_differentiation.
- [3] Boris Bolliet. *Research Computing and Software Development*. Accessed: 2024-12-12. 2024. URL: <https://researchcomputing.readthedocs.io/en/latest/index.html>.
- [4] Python Packaging Authority. *Writing pyproject.toml - A Full Example*. Accessed: 2024-12-12. n.d. URL: <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/#a-full-example>.
- [5] Boris Bolliet. *Company Package*. Accessed: 2024-12-12. 2024. URL: https://github.com/borisbolliet/company_package/tree/main.
- [6] Boris Bolliet. *Research Computing: Cython*. Accessed: 2024-12-12. 2024. URL: <https://researchcomputing.readthedocs.io/en/latest/material/part15/notebook.html>.

A Appendix

ChatGPT was used to help with resolving submodules issue that didn't allow for proper git tracking of folders inside the GitLab repository (suggested to use: `git rm --cached dual_autodiff_pkg`). ChatGPT was, also, used to help with formatting of this LATEX document, specifically with the following tasks:

- inserting an image into the document
- correcting math and code inputs to be represented in LATEX as desired
- using verbatim mode instead of 'texttt' for long code snippets
- creating citations
- suggesting 'verb' usage for filenames
- advising on proper numbering of citations (using [sorting=none])
- referencing another sentence from within the document

All AI usage for the package development is documented in the README.md file.